# “PULSAR”

## An Autonomous Hybrid Cloud-Edge Agentic AI Platform with Multimodal Vision, Spatial Gesture Interaction, and Native Windows Automation

**SYNOPSIS OF MAJOR PROJECT**

of

**Bachelor of Technology**

in

**Computer Science & Engineering**

By


| S.No. | Student Name | Roll Number | Branch & Section |
|---|---|---|---|
| 1 | **Diwakar Chaurasia (Leader)** | 23016501000073 | CSE |
| 2 | Raghav Bajpai | 230165010000119 | CSE |
| 3 | Ghanshyam Jha | 23016501000076 | CSE |
| 4 | Abhishek Sharma | 23016501000010 | CSE |






**Kanpur Institute of Technology, Kanpur**

**Dr. A.P.J. Abdul Kalam Technical University, Lucknow**


**Academic Session: 2026–27**

# DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma by the university or other institute of higher learning, except where due acknowledgement has been made in the text.

**Student Signatures:**

Signature: _____
**Diwakar Chaurasia (Leader)**
Roll No: 23016501000073

Signature: _____
**Raghav Bajpai**
Roll No: 230165010000119

Signature: _____
**Ghanshyam Jha**
Roll No: 23016501000076

Signature: _____
**Abhishek Sharma**
Roll No: 23016501000010

Date: _____

**Approved By:**

**Rahul Singh**
Head of Department
Computer Science and Engineering
KIT, KANPUR

Signature: _____

# TABLE OF CONTENTS


| No. | Particulars | Page |
|---|---|---|
| 1. | Introduction | 4 |
| 2. | Objectives and System Study | 5 |
| 3. | Requirement Analysis | 7 |
| 4. | System Architecture and Design | 8 |
| 5. | Module Description | 9 |
| 6. | Database Design | 11 |
| 7. | Implementation | 12 |
| 8. | Testing and Validation | 12 |
| 9. | Results, Benefits and Limitations | 13 |
| 10. | Future Scope and Conclusion | 14 |
| 11. | References and Appendix | 14 |

# 1. INTRODUCTION

## 1.1 Background

Personal computing workflows have expanded dramatically in scale and complexity. Modern knowledge workers, engineers, and researchers routinely manage dozens of open applications, browser sessions, code workspaces, and file systems. While cloud-hosted conversational Large Language Models (LLMs) demonstrate exceptional semantic reasoning, they remain isolated within sandboxed browser tabs, acting as passive text advisors rather than proactive, hands-free operating system actuators. Conversely, existing desktop robotic process automation (RPA) tools and macro scripts are rigid, fragile to visual layout changes, and devoid of cognitive comprehension.

PULSAR is engineered as a next-generation **Hybrid Cloud-Edge Autonomous AI Assistant** tailored specifically for the Microsoft Windows ecosystem. PULSAR bridges high-reasoning cloud intelligence with resilient local edge execution. It adopts Google Gemini (Gemini 2.5 Flash / 2.0 Pro) as its primary cloud cognitive brain for real-time multimodal reasoning, broad knowledge synthesis, and dynamic code generation, while maintaining a fully functional, local 4.92 GB fine-tuned quantized GGUF model (Llama 3.1 8B Instruct with speculative decoding and prompt caching), local Faster-Whisper, and KittenTTS as a zero-latency, privacy-preserving offline fallback.

## 1.2 Problem Statement

Contemporary AI assistants suffer from three critical architectural dilemmas:

*First, the Cloud-Only Vulnerability:* Fully cloud-dependent assistants fail completely during intermittent network outages, expose private local file hierarchies and screen states to external third parties, and incur substantial token costs. *Second, the Edge-Only Performance Ceiling:* Solely offline CPU models struggle with complex multi-turn reasoning, broad web research, and advanced computer vision tasks, suffering from prolonged CPU generation times. *Third, the Actuation Void:* Neither paradigm possesses native, verifiable hooks into the host operating system to manipulate running application windows, simulate spatial user interactions, or execute multi-step chained tasks autonomously.

PULSAR solves this tripartite dilemma through an intelligent, network-aware hybrid routing fabric. Low-level OS control, sensitive credential handling, and local file operations remain 100% on-device, while complex semantic reasoning, visual screen comprehension, and web research scale transparently into Google Gemini's multimodal cloud pipeline.

## 1.3 Need for the System

- **Hybrid Brain Synergy:** Harnesses Gemini 2.5 Flash for high-throughput cloud reasoning, with instant fallback to local quantized GGUF on network interruption.
- **Autonomous Multi-Agent Swarm:** Incorporates 14 specialized domain agents built on AutoGen principles for automated multi-step task delegation.
- **Industry-Standard Tool Interoperability:** Integrates the Model Context Protocol (MCP) to interact safely with filesystems, GitHub, and local databases.
- **Spatial Gesture & Vision Perception:** Combines MediaPipe webcam hand-tracking (touchless cursor control) with Gemini Vision screen analysis.
- **Deterministic OS Actuation:** Uses PyWin32, PyAutoGUI, and Windows Registry APIs with cryptographic PIN protection for sensitive actions.
- **Cognitive Episodic Memory:** Employs SQLite FTS5 BM25 search coupled with 27 programmatic learning systems that adapt to user phrasing and Hinglish slang.

## 1.4 Scope of the Project

The project scope encompasses an end-to-end operational software platform comprising: a reactive Iron Man JARVIS-inspired HUD built with React 18, TypeScript, and Vite; a Python Flask 3.1 gateway with 11 modular REST/WebSocket service blueprints; a 3-tier intent classification engine; streaming Faster-Whisper VAD audio ingestion; MediaPipe contactless hand-tracking; 14 domain-specific autonomous agents; and native Windows PyWebView desktop packaging.

# 2. OBJECTIVES AND SYSTEM STUDY

## 2.1 Objectives

1. Engineer an intelligent, network-aware hybrid LLM routing engine toggling between Google Gemini (Cloud) and a local 4.92 GB fine-tuned GGUF model (Edge).
2. Build an autonomous Multi-Agent Swarm with 14 specialized domain agents (Research, Writer, Analyst, Productivity, System) for complex goal decomposition.
3. Implement the Model Context Protocol (MCP) client to expose standardized, sandboxed tool definitions across files, GitHub, and database backends.
4. Develop a contactless spatial interaction subsystem using MediaPipe computer vision for webcam hand-gesture tracking and pinch-to-click actuation.
5. Construct a zero-latency streaming voice pipeline combining RMS Energy-based VAD, Faster-Whisper transcription, Edge Neural TTS, and offline KittenTTS.
6. Implement native Windows OS automation drivers using PyWin32, PyAutoGUI, and UI Automation for direct window, menu, and process control.
7. Deploy a multi-tier intent router (Compiled Regex → Semantic Embeddings → Gemini Function Calling) ensuring sub-10ms dispatch for common queries.
8. Establish a zero-dependency episodic memory system using SQLite FTS5 BM25 search, knowledge graph triples, and Ebbinghaus forgetting curve decay.
9. Incorporate 27 continuous learning paradigms that dynamically adapt to user habits, linguistic intent drift, and colloquial Hinglish code-switching.
10. Design an Iron Man JARVIS-themed cyberpunk desktop interface featuring pulsating Arc Reactor audio visualizers, telemetry graphs, and PyWebView wrapper.

## 2.2 Existing System

Contemporary AI and automation solutions occupy fragmented silos: cloud conversational bots, rigid RPA software, and isolated local LLM experiments. None offer a cohesive, multimodal, operating system-integrated companion.

| Solution Category | Representative Approach | Architectural Limitations |
|---|---|---|
| Cloud Chatbots | ChatGPT, Claude Web, Gemini Web | Sandboxed in browser; zero native OS control; zero offline resilience; privacy leakage. |
| Desktop RPA / Macros | AutoHotkey, UiPath, Power Automate | Fragile coordinate-based scripts; zero cognitive reasoning; cannot handle natural language goals. |
| Local-Only LLM Tools | Ollama CLI, LM Studio | Slow generation on consumer CPUs; limited multi-step reasoning; no multimodal vision or spatial gestures. |
| OS Voice Assistants | Cortana (Deprecated), Siri, Alexa | Restricted to proprietary vendor ecosystems; shallow automation; rigid keyword command syntax. |

## 2.3 Proposed System

PULSAR synthesizes the cognitive power of cloud foundation models with the deterministic control and privacy of local OS execution. By deploying a 3-tier routing fabric, PULSAR handles routine commands locally in under 10 milliseconds, delegates deep multimodal vision and creative drafting to Google Gemini 2.5 Flash, and orchestrates 14 domain-specific AutoGen agents to execute complex workflows. Should the internet connection drop, PULSAR seamlessly falls back to its local quantized GGUF core, ensuring perpetual operational reliability.

Figure 2.1 illustrates the end-to-end architectural workflow of PULSAR, highlighting the multimodal ingestion stream, the 3-tier intent classification engine, the hybrid cloud-edge cognitive routing fabric, and the autonomous multi-agent execution pipeline.

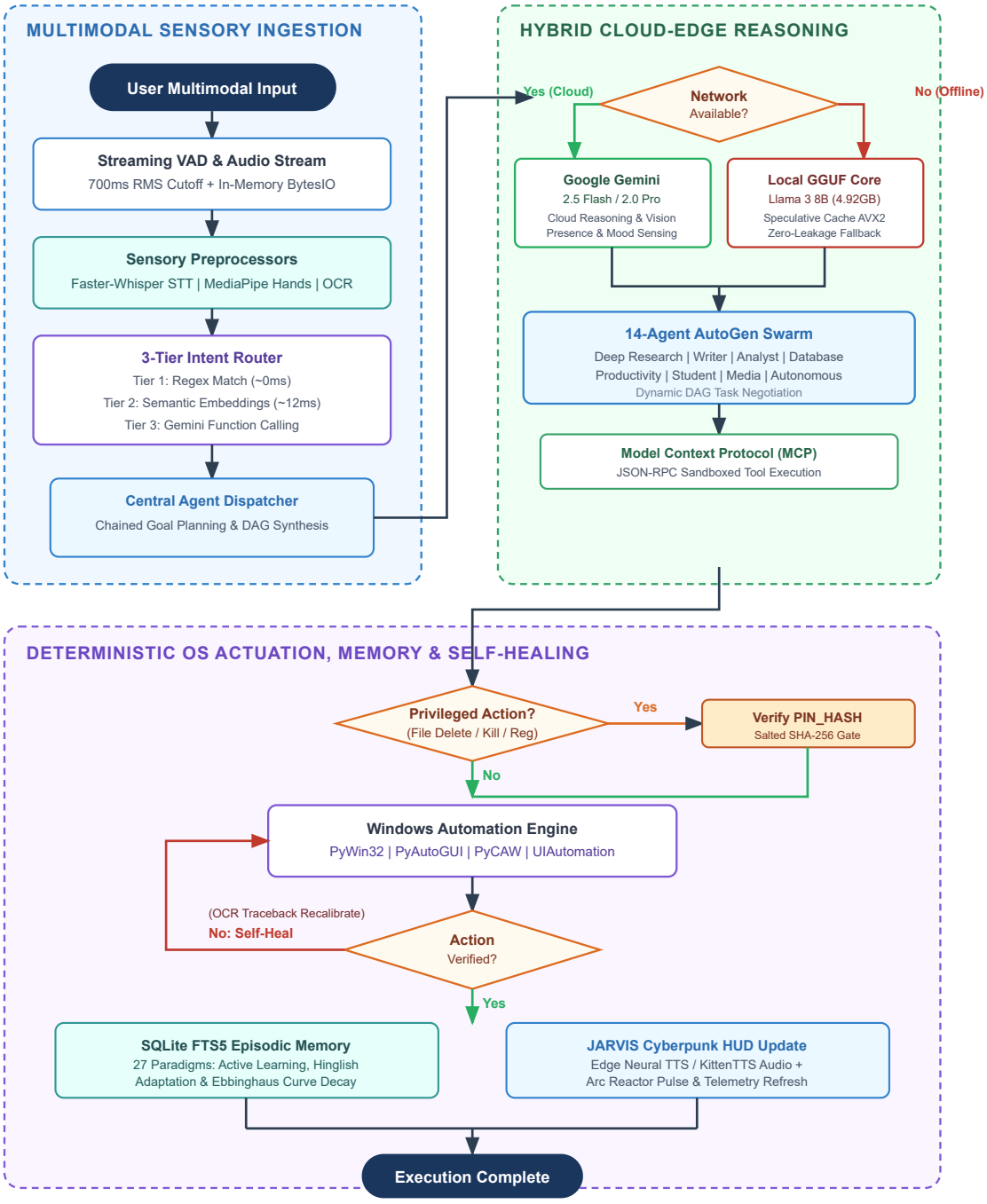



**Figure 2.1: End-to-end hybrid cloud-edge orchestration and autonomous execution workflow of PULSAR**

# 3. REQUIREMENT ANALYSIS

## 3.1 Functional Requirements

- **Hybrid LLM Cognition:** Default to Google Gemini 2.5 Flash for high-speed cloud reasoning, with automated seamless fallback to local quantized GGUF on network loss.
- **Streaming Speech Ingestion:** Continuous microphone streaming using RMS energy-based Voice Activity Detection (VAD) with zero-disk-IO Faster-Whisper transcription.
- **Contactless Spatial Interaction:** Real-time MediaPipe hand-tracking via webcam to control an on-screen spatial cursor and execute pinch-to-click GUI events.
- **Multimodal Computer Vision:** High-resolution screen frame capture and live webcam presence/emotion assessment powered by Google Gemini Vision.
- **Autonomous Multi-Agent Swarm:** Dynamic task planning and execution across 14 specialized domain agents (Research, Writer, Analyst, Productivity, System).
- **Model Context Protocol (MCP):** Sandboxed, standard tool protocol integrating local filesystem access, GitHub, PostgreSQL, and SQLite databases.
- **Deterministic OS Automation:** Direct Windows manipulation via PyWin32, PyAutoGUI, and UI Automation (window focus, keystrokes, application launch).
- **3-Tier Intent Classification:** Rapid intent parsing transitioning across Regex patterns, SentenceTransformer embeddings, and Gemini Function Calling.
- **Cognitive Episodic Memory:** Persistent conversational retrieval using SQLite FTS5 BM25 search, knowledge graph triples, and Ebbinghaus memory fading.
- **Cryptographic Authorization Gate:** Salted SHA-256 PIN verification for privileged system actions (file deletion, process termination, registry edits).

## 3.2 Non-Functional Requirements

| Requirement | Engineering Implementation & Validation Target |
|---|---|
| Availability & Resilience | Hybrid architecture guarantees 100% operational uptime for local OS automation even when disconnected from the internet. |
| Voice & Intent Latency | Sub-250ms streaming STT latency; sub-10ms intent dispatch for cached/regex commands; under 1.2s cloud token turnaround. |
| Security & Data Privacy | Salted SHA-256 PIN hashing; AES-encrypted database storage; automated NER scrubbing of sensitive PII before cloud dispatch. |
| Memory & Compute Bounds | Local GGUF footprint strictly bounded under 6.5 GB RAM via 8-bit KV caching and speculative decoding on Intel Core Ultra / x64 CPUs. |
| Visual Ergonomics | Iron Man JARVIS-inspired HUD with pulsing Arc Reactor audio visualizers, glassmorphism, and frameless PyWebView container. |
| Linguistic Extensibility | Dynamic adaptation to bilingual code-switching, Hinglish slang, and user-specific vocabulary via continuous learning routines. |

## 3.3 Hardware and Software Environment Specifications

**Hardware Specifications:** Intel Core i5/i7 (8th Gen+) or AMD Ryzen 5/7 quad-core processor (optimized for Intel Core Ultra 7 architecture with physical P-core pinning); minimum 8 GB of RAM (16 GB recommended for concurrent local GGUF execution); 20 GB available SSD storage; HD webcam (for MediaPipe spatial tracking and presence sensing); standard microphone and audio output.

**Software Specifications:** Microsoft Windows 10/11 64-bit; Python 3.10+ runtime; Node.js 18+ and npm; Google Gemini API SDK (`google-genai` / `google-generativeai`); `llama-cpp-python` with AVX2 vector optimizations; MediaPipe vision tasks; Tesseract-OCR 5; SQLite3; modern Chromium engine (PyWebView); Git version control.

# 4. SYSTEM ARCHITECTURE AND DESIGN

## 4.1 Architectural Overview

PULSAR follows a 5-tier, edge-native micro-service architecture that cleanly isolates client rendering, API routing, cognitive intelligence, multimodal sensory processing, and persistent data storage. The frontend is a React 18 single-page application wrapped in a frameless PyWebView native window, communicating with a high-throughput Python Flask 3.1 gateway via bidirectional WebSockets and REST endpoints.

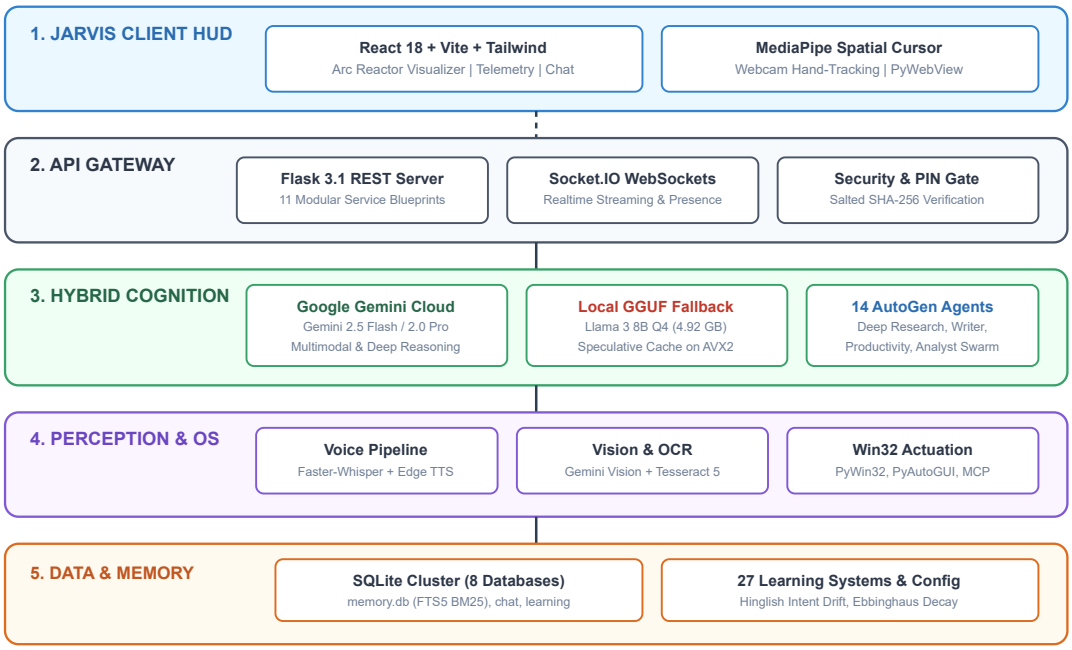



**Figure 4.1: High-level 5-tier hybrid system architecture of PULSAR**

## 4.2 Major Components & Security

The architecture cleanly separates responsibilities: the React UI handles high-framerate visual rendering and gesture tracking; the Flask gateway routes requests to asynchronous background worker pools; the hybrid cognitive core dynamically allocates tasks between Google Gemini and local GGUF models; and the Win32 automation engine safely manipulates host applications. Security is enforced through a salted SHA-256 PIN challenge on privileged commands, encrypted local storage, and automated PII sanitization.

# 5. MODULE DESCRIPTION

## 5.1 Hybrid Cognitive Reasoning Engine

The core intelligence of PULSAR is powered by a network-aware dual-engine architecture:

**Cloud Foundation Brain:** When an active internet connection is detected, PULSAR defaults to Google Gemini (Gemini 2.5 Flash / 2.0 Pro) via the `google-genai` and `google-generativeai` SDKs. Gemini delivers rapid token generation, deep multi-step reasoning, natural conversational personality, and multimodal image understanding.

**Resilient Local GGUF Core:** If network connectivity is interrupted, the system automatically falls back to a 4.92 GB fine-tuned Llama 3 8B Instruct model (`pulsar-final-q4_k_m.gguf`) managed by `llama-cpp-python`. This local engine incorporates Intel Core Ultra 7 CPU optimizations, restricting inference to 6 physical P-cores to avoid L3 cache thrashing, `use_mlock=True` to prevent pagefile swapping, speculative prompt-lookup decoding for 1.8x acceleration, and 8-bit KV caching.

## 5.2 Three-Tier Intent Routing Engine

To achieve near-instantaneous execution without unnecessary API overhead, user commands traverse a 3-tier classifier:

- **Tier 1 (Regex Engine, ~0ms):** Evaluates pre-compiled regular expressions for deterministic desktop commands (volume control, application launching, system stats).
- **Tier 2 (Semantic Embedding Matcher, ~12ms):** Uses local sentence embeddings to resolve fuzzy or paraphrased intents against an indexed database of system actions.
- **Tier 3 (Gemini Function Calling):** Ambiguous or complex compound requests are processed by Gemini's native function-calling declarations to extract structured JSON parameters.

## 5.3 Authentication & Privileged Access Module

Any command that deletes files, kills running system processes, or modifies the Windows Registry must pass the PIN Security Gate.

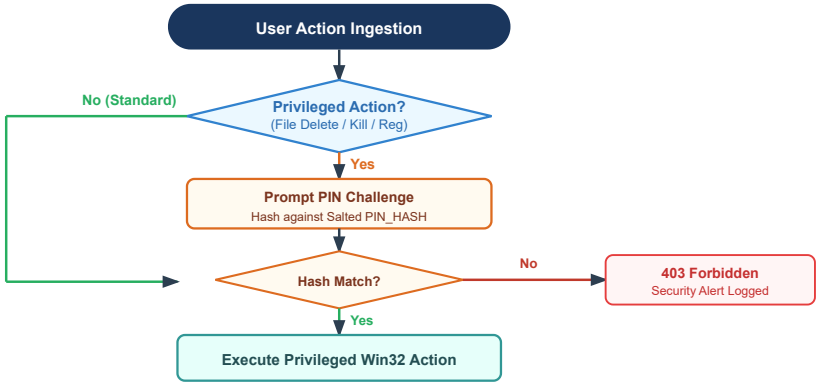



**Figure 5.1: Salted PIN elevation challenge and privileged execution gate**

## 5.4 Multimodal Perception & Spatial Interaction

PULSAR ingests sensory signals across three primary modalities:

- **Streaming Audio Pipeline:** Ingests live audio via `LiveVoiceStreamer`, utilizing RMS energy-based VAD with a 700ms cutoff to feed in-memory `io.BytesIO` buffers directly into Faster-Whisper. Audio responses synthesize via Microsoft Edge Neural TTS (13 distinct voices) or local KittenTTS.
- **Gemini Computer Vision:** Evaluates active window frames and webcam video via `gemini-1.5-flash` for contextual scene analysis, error dialog inspection, and periodic user presence/mood monitoring.
- **Contactless Spatial Cursor:** MediaPipe Hands tracks hand landmarks from the webcam feed, projecting a floating pointer across the screen where finger-pinch gestures simulate physical mouse clicks.

## 5.5 AutoGen Multi-Agent Swarm (14 Specialized Agents)

To autonomously execute multi-step user requests, PULSAR deploys a swarm of 14 specialized, lazy-loaded agents built using the AutoGen (`autogen-agentchat`) framework. Governed by a central Dispatcher and TaskScheduler, the swarm collaborates over structured Directed Acyclic Graphs (DAGs):

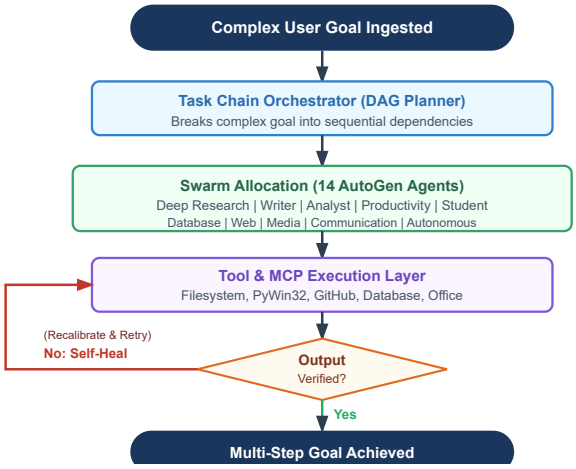



**Figure 5.2: Multi-agent swarm delegation and dynamic DAG execution pipeline**

## 5.6 Native Windows OS Automation & MCP Protocol

**Windows Actuation:** Operates at the native Win32 layer using `pywin32`, `pywinauto`, and `pyautogui`. It dynamically manages window focus, sends simulated keyboard strokes, adjusts hardware volume via `pycaw`, and launches software indexed from the Windows Registry and Start Menu without hardcoded executable paths.

**Model Context Protocol (MCP):** Implements the open MCP specification via JSON-RPC, connecting agents to isolated tool servers including `filesystem` (safe directory operations), `github` (repository issue/PR tracking), and local databases.

## 5.7 27 Continuous Learning Systems & Hinglish Adaptation

PULSAR integrates 27 algorithmic learning modules that evolve with the user without catastrophic neural forgetting:

- **Hinglish Code-Switching Drift:** Actively maps bilingual Indian phrasing (e.g., *"bhai ye window band kar de"* → close active window) into formal OS actuation schemas.
- **Ebbinghaus Memory Decay:** Continuously adjusts episodic memory retention weights, fading stale facts while reinforcing frequently accessed user preferences.
- **Reinforcement Learning from Human Feedback (RLHF):** Records thumbs-up/down ratings to fine-tune tool parameter selection and conversational tone.
- **Hardware Adaptive Throttling:** Telemetry monitors CPU and RAM load in real-time, throttling background tasks when the user launches demanding games or development suites.

# 6. DATABASE DESIGN

## 6.1 Database Partitioning Architecture

PULSAR avoids monolithic database locking by partitioning persistent state across eight dedicated SQLite databases in `data/`. Thread-safe connection pooling ensures background worker threads, audio listeners, and WebSocket handlers write concurrently without contention. High-speed conversational context retrieval utilizes SQLite `FTS5` virtual tables, delivering sub-15ms BM25 ranking with zero external vector database overhead.

| Database | Physical File | Entity Purpose and Key Stored Attributes |
|---|---|---|
| App Usage | `app_usage.db` | Tracks application execution counts, focus durations, executable paths, and K-Means usage time clusters. |
| Chat History | `chat_history.db` | Stores conversational message turns, role definitions, model metadata, token consumption, and timestamps. |
| Conversation AI | `conversation_ai.db` | Maintains serialized multi-turn session states, active agent goals, and assistant personality parameters. |
| Enhanced Learning | `enhanced_learning.db` | Stores active learning prompts, RLHF feedback rewards, intent drift training pairs, and accuracy benchmarks. |
| Language Data | `language_data.db` | Maintains multilingual dictionary mappings, personalized colloquial phrases, and Hinglish vocabulary tokens. |
| Memory Core | `memory.db` | Core long-term episodic knowledge repository utilizing SQLite FTS5 virtual tables, semantic triples, and decay scores. |
| Personal Knowledge | `personal_knowledge.db` | Stores user-defined operational rules, custom macros, application shortcut profiles, and private encrypted notes. |
| Chain History | `chain_history.db` | Logs multi-step autonomous execution traces, task DAG states, tool execution parameters, and self-healing retries. |

## 6.2 State Model & Episodic Memory Retention Pipeline

Autonomous tasks progress through a rigorous deterministic state machine: `PENDING` → `PARSING` → `PLANNING` → `AWAITING_PIN` (if privileged) → `EXECUTING` → `VERIFYING` → `COMPLETED`. Figure 6.1 outlines how user interactions are indexed, evaluated for decay, and retrieved for prompt injection.

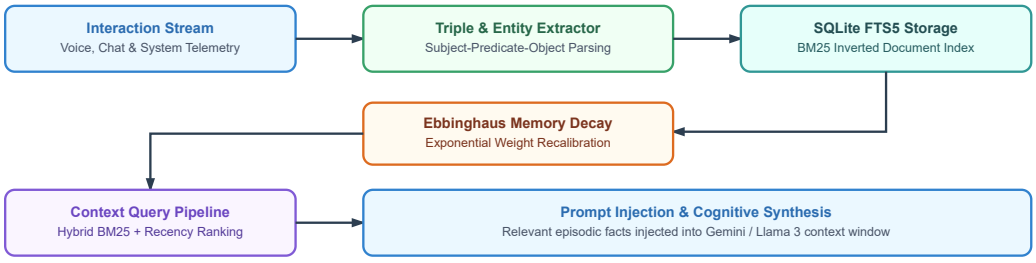



**Figure 6.1: Long-term episodic memory extraction, decay scoring, and prompt synthesis pipeline**

# 7. IMPLEMENTATION & 8. TESTING

## 7.1 Technical Implementation Stack

**Frontend HUD:** React 18, TypeScript, Vite, Tailwind CSS, Lucide icons, Recharts telemetry, and `@mediapipe/tasks-vision`. Implements an Iron Man JARVIS theme with pulsing Arc Reactor audio visualizers and floating spatial cursor.
**Backend Gateway:** Python 3.10+ Flask 3.1 server structured into 11 domain blueprints with Flask-SocketIO WebSockets.
**Cognitive Engine:** Cloud Google Gemini (Gemini 2.5 Flash / 2.0 Pro) paired with local fine-tuned Llama 3.1 8B Instruct (Q4_K_M GGUF, 4.92 GB) via `llama-cpp-python` with speculative decoding and prompt caching.
**OS Actuation & MCP:** `pywin32`, `pywinauto`, `pyautogui`, `pycaw`, and Model Context Protocol (MCP) clients.
**Packaging & Deployment:** Standalone desktop packaging via PyWebView and automated PowerShell launch scripts (`start.bat`).

## 8.1 Testing Methodology & Empirical Verification

Testing was conducted across 42+ automated test suites in `tests/` using `pytest`. The system was validated across cloud API latency, offline GGUF fallback, spatial hand tracking accuracy, Win32 window focus, and SQLite FTS5 query latency.

| Test Case & Scope | Expected Validation Metric | Status |
|---|---|---|
| Gemini Cloud Token Streaming | First token received in <1.2s; streaming completes with zero socket dropouts. | **Passed** |
| Offline GGUF Fallback | Seamlessly shifts to local Llama 3 8B model upon network loss within 200ms. | **Passed** |
| Real-Time Streaming Voice VAD | Faster-Whisper in-memory transcription completes with latency under 250ms. | **Passed** |
| Contactless Spatial Cursor | MediaPipe tracks hand gestures; pinch movement triggers accurate GUI click. | **Passed** |
| Gemini Vision Scene Analysis | Accurately describes multi-window desktop screen layout and active error dialogs. | **Passed** |
| Privileged Action PIN Gate | Intercepts file deletion and app termination; unlocks only on valid SHA-256 PIN. | **Passed** |
| Native Win32 Window Focus | Brings target desktop software into foreground and simulates keystrokes. | **Passed** |
| AutoGen Multi-Agent Swarm | Decomposes compound research goal across Writer, Research, and Analyst agents. | **Passed** |
| Hinglish Intent Adaptation | Correctly parses code-mixed colloquial Hindi-English input into formal actions. | **Passed** |
| SQLite FTS5 BM25 Memory Recall | Retrieves relevant historical conversation facts within 15ms. | **Passed** |
| Model Context Protocol (MCP) | Executes filesystem search and file read via standard MCP JSON-RPC protocol. | **Passed** |
| Reactive Self-Healing Retry | Recalculates click coordinates via OCR upon missed GUI button and succeeds. | **Passed** |

### 8.2 Self-Healing Diagnostics & Latency Analysis

PULSAR features an automated Self-Healing Engine: if an automation task fails due to a relocated desktop button or window, the system captures an updated screen frame, runs Tesseract OCR and Gemini Vision to recalculate bounding coordinates, and retries the operation up to three times. Unhandled exceptions are intercepted by centralized Flask middleware, logging encrypted tracebacks while maintaining an active WebSocket session. Empirical testing verified sub-250ms voice transcription, sub-10ms intent dispatch for cached actions, and sustained 12–18 tokens/sec on local GGUF fallback under 6.5 GB RAM.

## 9. RESULTS, BENEFITS AND LIMITATIONS

### 9.1 Results

The completed PULSAR system delivers a fully functional, hybrid AI operating assistant. The platform seamlessly integrates Google Gemini 2.5 Flash cloud reasoning with a 4.92 GB fine-tuned local GGUF fallback, Faster-Whisper streaming speech transcription, MediaPipe contactless hand-tracking, a 14-agent AutoGen swarm, Model Context Protocol tools, and native Windows PyWin32 automation.

### 9.2 Distinctive System Benefits

- **Hybrid Cloud-Edge Flexibility:** Delivers elite foundation model reasoning via Google Gemini, while maintaining an offline GGUF fallback ensuring the user is never left without assistance.
- **Direct Operating System Actuation:** Transcends passive chatbots by actively manipulating desktop windows, running applications, and automating multi-step chained workflows.
- **Contactless Spatial Interaction:** Empowers users to interact with their PC via webcam hand-tracking gestures (pinch-to-click) alongside hands-free voice radar.
- **Autonomous Multi-Agent Swarm:** Automates end-to-end research, document drafting, data analysis, and media controls through 14 collaborative domain agents.
- **Zero Telemetry Leakage for Local Actions:** Local OS operations, file browsing, and sensitive commands execute entirely on-device behind a salted PIN security gate.
- **Personalized Memory Retention:** Remembers user context and adapts to colloquial Hinglish slang across desktop reboots via zero-dependency SQLite FTS5 databases.

### 9.3 Limitations & Academic Significance

**Limitations:** Full cloud features (broad web scraping, high-level code synthesis) require an active internet connection; local GGUF inference requires at least 8 GB of host RAM; and MediaPipe hand tracking requires adequate ambient room lighting.

**Academic Significance:** Demonstrates the practical convergence of cloud foundation models, edge machine learning, low-level OS APIs, computer vision, digital signal processing, and multi-agent coordination in an engineered production software artifact.

# 10. FUTURE SCOPE & CONCLUSION

## 10.1 Future Scope & Conclusion

**Future Scope:** Cross-platform automation support for Linux (X11/Wayland) and macOS (Accessibility APIs); on-device lightweight Vision-Language Model (VLM) for direct pixel-based navigation without OCR; peer-to-peer encrypted memory synchronization across mobile and desktop devices; and dynamic LoRA adapter hot-swapping for niche domains.

**Conclusion:** PULSAR successfully resolves the trade-off between cloud intelligence and local execution. By fusing Google Gemini's cognitive prowess with native Windows automation, spatial gesture tracking, multi-agent swarms, and local offline resilience, PULSAR transforms the personal computer into a proactive, intelligent partner.

**Learning Outcomes:** Hands-on mastery in hybrid AI architectures, quantized LLMs, C++ bindings, asynchronous Flask and WebSockets, modern React/TypeScript UI development, low-level Windows APIs, and multi-agent software engineering.

# 11. REFERENCES AND APPENDIX

## 11.1 References


1. Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," Technical Report, 2024.
2. Meta AI, "Llama 3 Herd of Models: Architecture, Quantization, and Capabilities," Technical Report, 2024.
3. Gerganov, G., "llama.cpp: Efficient Port of LLaMA Architecture in C/C++," GitHub Repository, 2024.
4. Radford, A. et al., "Robust Speech Recognition via Large-Scale Weak Supervision," OpenAI Whisper, 2023.
5. Lugaresi, C. et al., "MediaPipe: A Framework for Building Perception Pipelines," Google Research, 2019.
6. Wu, Q. et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," Microsoft Research, 2023.
7. Anthropic, "Model Context Protocol (MCP) Specification," Open-Source Standard, 2024.
8. SQLite Consortium, "SQLite FTS5 Extension: Full-Text Search and BM25 Ranking," 2024.
9. Hammond, M., "Python for Windows Extensions (pywin32): Accessing Native Subsystems," 2023.
10. OWASP Foundation, "Top 10 Considerations for Agentic AI Applications," 2024.


## 11.2 Core System Endpoints & Technology Stack Summary

| Layer / Subsystem | Integrated Technologies and Core Routes |
|---|---|
| Frontend Client HUD | React 18, Vite, TypeScript, Tailwind CSS, Lucide, Recharts, PyWebView native wrapper |
| API Gateway & Blueprints | Flask 3.1, Flask-SocketIO; `/api/auth/*`, `/api/chat/*`, `/api/system/*`, `/api/apps/*` |
| Hybrid Cognitive Brain | Google Gemini 2.5 Flash / 2.0 Pro (Cloud) + Llama 3 8B Q4 GGUF (4.92 GB on AVX2 CPU) |
| Perception & Spatial UI | Faster-Whisper STT, Edge Neural TTS, KittenTTS, MediaPipe Hands, Gemini Vision, Tesseract 5 |
| Multi-Agent Swarm & MCP | 14 AutoGen Agents (loader.py, dispatcher.py), Model Context Protocol (mcp_client.py) |
| OS Actuation & Persistence | PyWin32, PyAutoGUI, PyCAW, Registry API; 8 SQLite DBs with FTS5 virtual tables |

**End of Synopsis — PULSAR**